

easyRef 蓝牙数据接口

- [1 视力筛查设备软件要求](#)
- [2 蓝牙连接示例代码](#)
 - [2.1 蓝牙连接](#)
 - [2.2 回调接口](#)
 - [2.3 调用](#)
- [3 蓝牙配对](#)
- [4 通讯协议](#)

1 视力筛查设备软件要求

要求软件版本 >= 1.0.68010.2878

2 蓝牙连接示例代码

2.1 蓝牙连接

仅供广州开视医疗科技有限公司使用
(深圳莫廷医疗科技有限公司)

```

public class MptBleManager {

    private static final UUID BluetoothUUID = UUID.fromString("00001101-0000-1000-8000-00805F9B34FB");

    private final static String TAG = MptBleManager.class.getSimpleName();

    private Context mContext;

    private BluetoothAdapter mBluetoothAdapter;
    private BLECallback mBluetoothCallback;
    private BLEConnectThread bleConnectThread;

    private final static int MSG_RESULT = 0;
    private final static int MSG_ERROR = 1;
    private final static int MSG_CONNECT = 2;
    private final static int MSG_CLOSE = 3;

    @SuppressWarnings("HandlerLeak")
    private final Handler myHandler = new Handler() {
        @Override
        public void handleMessage(Message msg) {
            super.handleMessage(msg);
            switch (msg.what) {
                case MSG_RESULT:
                    if (mBluetoothCallback != null){
                        mBluetoothCallback.onResult((String) msg.obj);
                    }
                    break;
                case MSG_ERROR:
                    if(mBluetoothCallback != null){
                        mBluetoothCallback.onError((String) msg.obj);
                    }
                    break;
                case MSG_CONNECT:
                    if(mBluetoothCallback != null){
                        mBluetoothCallback.onConnect();
                    }
                    break;
                case MSG_CLOSE:
                    //TODO bleConnectThread = null
                    //
                    if(mBluetoothCallback != null){
                        mBluetoothCallback.onClose();
                    }
                    break;
            }
        }
    };

    public MptBleManager(Context context) {
        mContext = context;
        mBluetoothAdapter = BluetoothAdapter.getDefaultAdapter();
    }

    public void setCallback(BLECallback callback){
        mBluetoothCallback = callback;
    }

    public void connect(BluetoothDevice device){
        if (mBluetoothAdapter == null) {
            String errMsg = "Get bluetooth adapter fail!";
            Log.e(TAG, errMsg);
            myHandler.sendMessage(Message.obtain(myHandler, MSG_ERROR, errMsg));
            return;
        }

        if(bleConnectThread == null){
            bleConnectThread = new BLEConnectThread(device);
            bleConnectThread.start();
        }
    }

    public void disconnect(){
        if(bleConnectThread != null){
            bleConnectThread.disconnect();
            try {
                Log.w(TAG, "disconnect: try join");
                bleConnectThread.join();
            }
        }
    }
}

```

```

        Log.w(TAG, "disconnect: end join");
        bleConnectThread = null;
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
}

private class BLEConnectThread extends Thread {

    private BluetoothDevice bluetoothDevice;
    private BluetoothSocket bluetoothSocket;

    private final String mHeartBeatMsg = "-^--^--";
    private long mHeartBeatDate;

    private final Object lock = new Object();

    BLEConnectThread(BluetoothDevice device) {
        bluetoothDevice = device;
    }

    @Override
    public void run() {

        while (bluetoothSocket == null){
            try {
                bluetoothSocket = bluetoothDevice.createInsecureRfcommSocketToServiceRecord(BluetoothUUID);
            } catch (Exception e) {
                Log.d(TAG, "UUID 创建Socket异常 " + e.getMessage());
            }

            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }

        //连接线程
        new Thread() {
            @Override
            public void run() {
                try {
                    synchronized (lock) {
                        bluetoothSocket.connect();

                        //数据接收线程
                        new Thread(new Runnable() {
                            @Override
                            public void run() {
                                try {
                                    while (bluetoothSocket != null && bluetoothSocket.isConnected()) {
                                        InputStream inputStream = bluetoothSocket.getInputStream();
                                        int count = 0;
                                        while (count == 0) {
                                            count = inputStream.available();
                                        }
                                        final byte[] buf = new byte[count];

                                        final int ret = inputStream.read(buf);
                                        Log.d(TAG, "read ret " + ret);

                                        final String data = new String(buf, StandardCharsets.UTF_8);
                                        Log.d(TAG, "read data " + data);

                                        if(mHeartBeatMsg.equals(data)){
                                            mHeartBeatDate = System.currentTimeMillis();
                                        }else{
                                            myHandler.sendMessage(Message.obtain(myHandler, MSG_RESULT, data));
                                        }
                                    }
                                } catch (IOException e) {
                                    Log.e(TAG, "ble read err: " + e.getMessage());
                                }
                            }
                        }).start();
                    }
                    myHandler.sendMessage(Message.obtain(myHandler, MSG_CLOSE));
                    Log.d(TAG, "run--3--: BLE read thread finish");
                }
            }
        }.start();
    }
}

```



```
//蓝牙权限申请完成后
MptBleManager mMptBleManager = new MptBleManager(MainActivity.this);
mMptBleManager.setCallback(new BLECallback() {
    @Override
    public void onConnect() {
    }

    @Override
    public void onError(String errorMsg) {
    }

    @Override
    public void onResult(String result) {
    }

    @Override
    public void onClose() {
    }
});

//视力筛查设备蓝牙名称已“easyRef-”开头
mMptBleManager.connect(BluetoothDevice);
```

3 蓝牙配对

代码中执行蓝牙连接后，如果是首次连接会弹出配对窗口

蓝牙首次配对正常情况下会弹出请求框，如下图，点击允许即可



4 通讯协议

1. 蓝牙连接心跳包

心跳包频率 5秒/次，心跳包内容长度固定为5，内容为 -^--^-

2. 视力筛查结果

每测一次完成后发送结果, 为JSON格式 例如:

```
{
  "A": "153°",
  "C": "-0.75D",
  "S": "-0.25D",
  "deviceName": "easyRef-4PQL2Q",
  "examID": "20211122112546",
  "resultTag": "R-Avg"
}
```

参数描述:

参数	类型	描述
examID	String	测试编号，一个编号对应一组左右眼测试
deviceName	String	设备名称，和蓝牙名称相同
resultTag	String	代表左眼或右眼的结果
S	String	球镜度
C	String	柱镜度
A	String	轴位

仅供广州开视医疗科技有限公司使用
(深圳莫廷医疗科技有限公司)